

ENAE644 Term Project

Vai Srivastava¹

Abstract—This project implements and evaluates adversarial motion planning algorithms in a two-agent scenario where a deceptive agent attempts to reach a hidden goal while concealing its intent, and an interceptor agent seeks to infer the hidden goal and intercept the deceptive agent. The deceptive agent employs Adversarial RRT*, a sampling-based planner that extends RRT* by incorporating a learned deception cost function. A recurrent neural network serves as a surrogate observer, and the planner balances path optimality against observer classification accuracy using a weighted cost function. The interceptor agent combines inverse reinforcement learning to recover a behavioral model of the deceptive agent from historical demonstrations, particle filtering for online belief distribution tracking over candidate goals, and game-theoretic model predictive control for real-time interception planning with replanning. Both agents operate in a continuous two-dimensional workspace and are implemented in Python using JAX and Equinox for differentiable programming and GPU acceleration. Comprehensive software specifications have been developed, detailing the agent-based architecture, algorithm pseudocode, data schemas, and JAX implementation patterns.

The shared infrastructure implementation—including workspace representation with obstacle modeling, collision detection primitives using ray casting and vectorized checking, trajectory handling as JAX pytrees with interpolation, and agent controller interfaces—has been completed with accompanying test coverage. The simulation framework is partially implemented, with visualization, orchestration, and configuration systems in place. Remaining work focuses on neural network training (RNN observer and IRL behavioral model) and agent-specific planning algorithms (Adversarial RRT*, particle filter, game-theoretic MPC).

NOTE: Experimental results and performance analysis will be added upon completion of implementation and evaluation. Planned experiments include baseline performance assessment, neural network training and validation, full adversarial scenarios with parameter sweeps, and ablation studies to isolate component contributions.

I. INTRODUCTION

The goal of this project is to implement and evaluate algorithms from two complementary sub-fields of adversarial motion planning. The first is *deceptive motion planning*, which is concerned with generating trajectories that conceal the

agent’s true objective from an observer [1], [2]. The second is *goal identification and interception*, which seeks to infer an observed agent’s hidden objective and plan a trajectory to intercept it [3], [4]. Rather than studying these in isolation, the project will pit a deceptive planner against an interception-focused planner in a shared environment, providing a direct adversarial evaluation of each algorithm’s strengths and limitations. Both agents will employ planning methods at least as sophisticated as RRT*, incorporating learning-based and game-theoretic components on top of their respective base planners.

II. MOTIVATION

Deceptive motion planning has direct applications in video game AI, where non-player characters must exhibit unpredictable yet purposeful behavior [3], [4]. However, the relevance of this problem extends well beyond entertainment into real-world domains such as strategic logistics, privacy-preserving navigation, and military operations. The ability to move deceptively can be the difference between mission success and failure [1], [2], while the complementary problem of identifying and intercepting deceptive agents is equally critical: security and defense systems must be capable of recognizing when an observed agent is behaving deceptively and responding accordingly [3], [5]. Studying the interplay between deception and counter-deception will provide insight into the robustness of each approach and reveal failure modes that would not surface when evaluating either in isolation.

It is important to note that prior work has explored this coupling in the context of autonomous vehicles [6] and multi-agent navigation [7], [8], but an adversarial pitting in a strategy-game context remains underexplored.

III. RELATED WORK

A. Deceptive Motion Planning

A central challenge in deceptive motion planning is balancing path cost against unpredictability. Nichols et al. [1] address this trade-off with Adversarial RRT*, a sampling-based planner that incorporates a learned deception cost into the RRT* objective. Their approach uses a recurrent neural network as a surrogate observer and penalizes trajectories that the network can classify correctly, achieving low observer accuracy while keeping path length near-optimal. Xu et al. [2] take a complementary approach grounded in probabilistic goal

¹Srivastava is with the Department of Aerospace Engineering, University of Maryland, College Park, MD 20742, The United States of America, vaisriv@umd.edu.

recognition, formalizing deceptive path planning as a mixed-integer program that maximizes goal ambiguity with respect to an observer's belief distribution. Both methods assume a passive observer and evaluate deception in isolation rather than against an active adversary, which is the gap this project aims to address.

B. Goal Recognition and Interception

On the opposing side of the problem, several works focus on inferring an agent's hidden objective and acting on that inference. Xu and Yin [5] introduce *relative goal uncertainty*, an entropy-based metric that quantifies the goal-related information contained in each action, and show how it can be used to control the goal identification process from either side of the interaction. Zeng and Xu [3] extend goal recognition to explicitly account for deceptive agents by using inverse reinforcement learning to recover a behavioral model from the deceiver's historical data, then constructing a game-theoretic interference strategy. In the domain of first-person shooter games, Tastan et al. [4] learn player-specific motion models via inverse reinforcement learning and use particle-filter-based prediction to intercept opponents on partially occluded maps. These works provide the algorithmic foundations for the interceptor agent in this project, particularly the combination of learned behavioral models with online prediction.

C. Game-Theoretic and Interactive Planning

A related body of work frames multi-agent planning as a dynamic game in which each agent's strategy depends on the others'. Le Cleac'h et al. [8] propose LUCIDGames, which pairs an unscented Kalman filter for online estimation of other agents' cost functions with a receding-horizon game-theoretic planner, demonstrating real-time performance in autonomous driving scenarios. Espinoza et al. [6] tightly couple prediction and planning through a game-theoretic MPC that uses a multi-agent neural network policy as its predictive model, producing interactive behaviors between an autonomous vehicle and surrounding traffic. While both of these works operate in the autonomous driving domain, they demonstrate the value of jointly reasoning about one's own plan and other agents' likely responses, directly informing the adversarial evaluation framework in this project.

D. Adversary-Aware Motion Planning

Most closely related to the present work, Netter and Vamvoudakis [7] propose a motion planning framework in which a player agent navigates a multi-agent environment while simultaneously identifying and avoiding potential adversaries using Gaussian process classification. Their method includes real-time replanning to avoid likely adversarial agents and distinguishes adversaries from benign agents to prevent unnecessary evasive maneuvers. This project builds on a similar premise but inverts the emphasis: rather than treating

interception avoidance as a byproduct of adversary classification, the deceptive agent here will actively minimize observer accuracy as a first-class planning objective, while the interceptor will be a fully autonomous adversary rather than a fixed behavioral model.

IV. FORMAL PROBLEM DEFINITION

This section formalizes the adversarial motion planning problem as a two-agent scenario in which a deceptive agent attempts to reach a hidden goal while concealing its intent, and an interceptor agent seeks to infer the hidden goal and intercept the deceptive agent before the goal is reached.

A. Workspace and Agent Models

We consider a bounded, continuous two-dimensional workspace $\mathcal{W} \subset \mathbb{R}^2$ containing a static obstacle set $\mathcal{O} \subset \mathcal{W}$. The collision-free workspace is defined as $\mathcal{W}_{\text{free}} = \mathcal{W} \setminus \mathcal{O}$. Two agents operate within this workspace: a deceptive agent (Agent D) and an interceptor agent (Agent I).

1) *Deceptive Agent (Agent D)*: The deceptive agent's state at time t is its position $x_D(t) \in \mathcal{W}_{\text{free}}$. Agent D has a finite set of candidate goals $\mathcal{G}_D = \{g_1, g_2, \dots, g_M\} \subset \mathcal{W}_{\text{free}}$ known to both agents, and a true goal $g^* \in \mathcal{G}_D$ that is hidden from Agent I. The agent executes a trajectory $\xi_D : [0, T] \rightarrow \mathcal{W}_{\text{free}}$ satisfying the boundary conditions $\xi_D(0) = x_{D,0}$ and $\xi_D(T) = g^*$, where $x_{D,0}$ is the initial position and T is the time horizon. The trajectory is subject to kinodynamic constraints including velocity bounds and acceleration limits, which we denote generically as $\xi_D \in \Xi_D$, where Ξ_D is the set of kinodynamically feasible trajectories.

2) *Interceptor Agent (Agent I)*: The interceptor agent's state at time t is its position $x_I(t) \in \mathcal{W}_{\text{free}}$. Agent I starts at position $x_{I,0}$ and executes a trajectory $\xi_I : [0, T] \rightarrow \mathcal{W}_{\text{free}}$ with $\xi_I(0) = x_{I,0}$. The objective of Agent I is to infer the true goal g^* from observations of Agent D's partial trajectory and plan an interception trajectory to reach the predicted goal before Agent D. Agent I's trajectory is also subject to kinodynamic constraints $\xi_I \in \Xi_I$, which may differ from those of Agent D.

3) *Observer Model*: Agent I maintains a belief distribution over the candidate goals, represented as $b_t : \mathcal{G}_D \rightarrow [0, 1]$ where $\sum_{g \in \mathcal{G}_D} b_t(g) = 1$. This belief is updated incrementally based on observations of the partial trajectory $\xi_D[0, t]$. The belief update mechanism will be specified in detail in the following subsections.

B. Deceptive Motion Planning Problem

The deceptive agent seeks to generate a trajectory ξ_D^* that satisfies three competing objectives: (1) reach the true goal g^* , (2) minimize path cost, and (3) maximize the observer's uncertainty about g^* . This multi-objective optimization problem is formalized as

$$\xi_D^* = \arg \min_{\xi_D \in \Xi_D} [\alpha \cdot J_{\text{path}}(\xi_D) + (1 - \alpha) \cdot J_{\text{deception}}(\xi_D)] \quad (1)$$

where $J_{\text{path}}(\xi_D)$ is the path cost, $J_{\text{deception}}(\xi_D)$ quantifies the “revealingness” of the trajectory (i.e., how easily an observer can infer the true goal), and $\alpha \in [0, 1]$ is a weighting parameter that balances path optimality against deception.

The path cost is typically defined as the total path length or control effort:

$$J_{\text{path}}(\xi_D) = \int_0^T \|\dot{\xi}_D(t)\| dt \quad (2)$$

The deception cost is more subtle and can be formulated in multiple ways. Following the Adversarial RRT* framework [1], we employ a learned classifier f_θ (specifically, a recurrent neural network) as a surrogate observer. The classifier maps partial trajectories to probability distributions over goals: $f_\theta : \xi_D \mapsto \Delta(\mathcal{G}_D)$, where $\Delta(\mathcal{G}_D)$ denotes the probability simplex over $\mathcal{G}_D$. The deception cost can then be defined as the negative entropy of the classifier’s output:

$$J_{\text{deception}}(\xi_D) = -H(f_\theta(\xi_D)) \quad (3)$$

where $H(p) = -\sum_{g \in \mathcal{G}_D} p(g) \log p(g)$ is the Shannon entropy. High entropy corresponds to high uncertainty (good deception), so minimizing $-H$ is equivalent to maximizing uncertainty. Alternatively, the deception cost can be defined directly as the classifier’s accuracy:

$$J_{\text{deception}}(\xi_D) = P_\theta(\text{correct goal} \mid \xi_D) = f_\theta(\xi_D)(g^*) \quad (4)$$

An alternative formulation based on probabilistic goal recognition [2], [5] defines the deception cost as the negative goal ambiguity:

$$J_{\text{deception}}(\xi_D) = -\sum_{g \in \mathcal{G}_D} P(g \mid \xi_D) \log P(g \mid \xi_D) \quad (5)$$

where the posterior probability $P(g \mid \xi_D)$ is computed using a probabilistic goal recognition framework that models the likelihood of trajectories under different goal hypotheses.

The optimization problem is subject to the following constraints:

- Collision avoidance: $\xi_D(t) \in \mathcal{W}_{\text{free}}$ for all $t \in [0, T]$
- Kinodynamic feasibility: $\xi_D \in \Xi_D$ (velocity and acceleration limits)
- Boundary conditions: $\xi_D(0) = x_{D,0}$ and $\xi_D(T) = g^*$

1) *Solution Method*: We employ Adversarial RRT* [1], a sampling-based motion planner that extends the RRT* algorithm by incorporating the deception cost $J_{\text{deception}}$ into the cost function used for tree rewiring and path selection. The planner iteratively samples configurations, extends the search tree, and evaluates candidate paths using the combined cost $\alpha J_{\text{path}} + (1 - \alpha) J_{\text{deception}}$, producing trajectories that balance path optimality with unpredictability.

C. Goal Inference and Interception Problem

The interceptor agent must solve two interrelated subproblems:

- 1) Infer the deceptive agent’s true goal g^* from observations of the partial trajectory $\xi_D[0, t]$
- 2) Plan an interception trajectory to reach the predicted goal before Agent D.

1) *Goal Inference Subproblem*: The goal inference problem is addressed through a combination of offline learning and online belief updates.

a) *Inverse Reinforcement Learning (IRL) Phase*: Prior to runtime, Agent I uses inverse reinforcement learning to recover a behavioral model of Agent D from historical trajectory data. The IRL problem seeks to recover a reward (or cost) function $R_D(s, a)$ such that the observed trajectories are approximately optimal under this reward function. Following [3], we model the deceptive agent’s behavior by learning a reward function that captures both path efficiency and deception, enabling Agent I to predict likely future actions given a goal hypothesis.

b) *Online Belief Update*: During execution, Agent I maintains and updates the belief distribution $b_t(g)$ over candidate goals using Bayes’ rule:

$$b_{t+1}(g) \propto P(\xi_D[t, t+1] \mid g, \xi_D[0, t]) \cdot b_t(g) \quad (6)$$

where $P(\xi_D[t, t+1] \mid g, \xi_D[0, t])$ is the likelihood of the observed trajectory segment given goal hypothesis g and the trajectory history. This likelihood is computed using the behavioral model learned via IRL.

c) *Particle Filter Implementation*: Following [4], we employ a particle filter to maintain and propagate the belief distribution. Each particle represents a hypothesis about the true goal and the agent’s current state. The particle filter alternates between a prediction step (using the learned motion model to propagate particles forward) and an update step (reweighting particles based on observed trajectory segments).

The predicted goal at time t is obtained by selecting the maximum a posteriori (MAP) estimate:

$$\hat{g}(t) = \arg \max_{g \in \mathcal{G}_D} b_t(g) \quad (7)$$

2) *Interception Planning Subproblem*: Given the predicted goal $\hat{g}(t)$, Agent I must plan a trajectory to intercept Agent D. The interception planning problem is formulated as:

$$\xi_I^* = \arg \min_{\xi_I \in \Xi_I} T_{\text{intercept}} \quad (8)$$

subject to the constraint that Agent I reaches the predicted goal $\hat{g}$ before Agent D. However, because the prediction $\hat{g}(t)$ evolves over time as new observations are incorporated, a static plan is insufficient.

a) *Game-Theoretic Model Predictive Control (MPC)*: Following [8] and [6], we employ a receding-horizon game-theoretic planner. At each time step t , Agent I solves the finite-horizon optimization problem:

$$\min_{\xi_I[t, t+H]} J_I(\xi_I, \hat{\xi}_D) \quad (9)$$

where H is the prediction horizon and $\hat{\xi}_D$ is the predicted future trajectory of Agent D, computed by forward-simulating the learned behavioral model conditioned on the current belief distribution. The cost function J_I is defined as:

$$J_I(\xi_I, \hat{\xi}_D) = \|\xi_I(T_{\text{intercept}}) - \hat{g}\|^2 + \int_t^{t+H} \|\dot{\xi}_I(\tau)\|^2 d\tau \quad (10)$$

where the first term penalizes distance to the predicted goal and the second term penalizes control effort. The solution yields a planned trajectory for the next H time steps, of which only the first segment is executed before replanning at the next time step.

b) Replanning: The MPC framework enables real-time replanning. As Agent I observes new segments of Agent D's trajectory, the belief distribution $b_t(g)$ is updated, the predicted goal $\hat{g}(t)$ may change, and the interception plan is recomputed accordingly.

D. Adversarial Game Formulation

The interaction between the deceptive agent and the interceptor can be formalized as a two-player dynamic game with asymmetric information.

1) Players and Objectives:

- Player 1 (Agent D): Minimizes $J_D = \alpha J_{\text{path}} + (1 - \alpha) J_{\text{deception}}$
- Player 2 (Agent I): Minimizes J_I (interception cost, including time-to-intercept and control effort)

2) Information Structure: This is an asymmetric information game. Agent D has complete information: it knows the true goal g^* , the candidate goal set $\mathcal{G}_D$, and can observe Agent I's current state $x_I(t)$. In contrast, Agent I has incomplete information: it knows the candidate goal set $\mathcal{G}_D$ and can observe the partial trajectory $\xi_D[0, t]$, but does not know the true goal g^* . Agent I must infer g^* from observations.

3) Solution Concept: Due to the information asymmetry and the sequential nature of the interaction, this problem does not admit a standard Nash equilibrium solution. Instead, it is more appropriately modeled as a Stackelberg game (leader-follower game), in which Agent D (the leader) plans its trajectory anticipating that Agent I (the follower) will respond optimally given its observations. However, Agent D must account for the fact that Agent I does not know g^* and will update its beliefs and replan based on observed trajectory segments.

a) Adversarial Evaluation Framework: Rather than solving analytically for an equilibrium, we adopt an empirical adversarial evaluation framework. We implement Adversarial RRT* for Agent D and the IRL-based particle filter with game-theoretic MPC for Agent I, then evaluate their performance through simulation. Key performance metrics include:

- For Agent D: Observer classification accuracy at the time of goal completion, $P_\theta(g^* | \xi_D)$, and path length ratio $J_{\text{path}}(\xi_D^*)/J_{\text{path}}(\xi_D^{\text{opt}})$ where ξ_D^{opt} is the optimal (shortest) path
- For Agent I: Goal inference accuracy $\mathbb{1}[\hat{g}(T) = g^*]$ and time-to-intercept relative to Agent D's arrival time
- Overall: Success rate for each agent—does Agent D successfully reach g^* before being intercepted, or does Agent I successfully intercept Agent D?

b) Interaction Dynamics: The game proceeds as follows.

Agent D generates a deceptive trajectory using Adversarial RRT*. Agent I observes the trajectory incrementally and updates its belief distribution over goals using the IRL-learned behavioral model and particle filtering. Based on the updated belief, Agent I replans its interception trajectory using game-theoretic MPC. This cycle continues until one of two termination conditions is met: (a) Agent D reaches the true goal g^* , or (b) Agent I successfully intercepts Agent D (i.e., $\|x_I(t) - x_D(t)\| < \varepsilon_{\text{intercept}}$ for some small threshold $\varepsilon_{\text{intercept}}$).

V. METHODOLOGY

A. Overview

The project is implemented in Python, chosen for its extensive ecosystem of machine learning and scientific computing libraries. In particular, we use JAX (via the Equinox library) as the primary framework for differentiable programming, automatic differentiation, and GPU-accelerated computation.

The core of the project involves two competing agents in a continuous two-dimensional workspace. The first agent employs a deceptive motion planning algorithm building on Adversarial RRT* [1], which augments the sampling-based RRT* planner with a learned deception cost to generate trajectories that minimize an observer's ability to infer the agent's true goal. Entropy-based deceptive planning techniques [2], [5] inform the design of the deception objective. The second agent utilizes an identification and interception strategy, drawing on techniques from inverse reinforcement learning and game-theoretic prediction [3], [4], [8], with the objective of recognizing the deceptive agent's true goal and planning an intercept trajectory in real time. The approach of coupling adversary identification with reactive planning follows the framework proposed by Netter and Vamvoudakis [7], adapted to the adversarial evaluation setting. Both agents thus operate well beyond basic shortest-path planning: the deceptive agent layers a learned deception metric on top of RRT*, while the interceptor combines online goal inference with reactive replanning.

Of note, despite several of the referenced algorithms being originally developed for three-dimensional environments (e.g., first-person shooter game maps [4]), we work in a continuous two-dimensional workspace for ease of development and in

the interest of completing the project within the semester. The simulation environment is custom-built in Python, providing full control over the experimental setup and enabling systematic evaluation of each algorithm’s performance under varying conditions.

B. Implementation

Implementation has progressed through the first major phase. The comprehensive software specifications (documented in `./docs/spec/`) have been completed, and development of the shared infrastructure has been finalized. The simulation framework is currently in progress, with orchestration and visualization components complete. The implementation follows a functional programming approach using JAX for automatic differentiation and GPU acceleration, with Equinox for neural network components.

The shared infrastructure package (`src/shared/`) is fully implemented and operational. The workspace module (`workspace.py`) provides a 2D bounded workspace representation with obstacle modeling, supporting both circular obstacles (defined by center and radius) and arbitrary convex polygon obstacles (defined by vertex lists). Workspace validation includes bounds checking and collision-free verification using rejection sampling for random point generation. The collision detection module (`collision.py`) implements JIT-compiled primitives including point-in-circle tests, point-in-polygon detection via ray casting algorithm, segment-circle intersection using point-to-segment distance, segment-polygon collision with edge intersection checking, and vectorized batch collision checking using JAX’s `vmap` for parallel processing. The geometry module (`geometry.py`) supplies core geometric utilities: Euclidean and squared distance calculations, angle computations (both between vectors and from point to point using `atan2`), vector normalization and rotation, point-to-segment projection with parameter clamping, 2D cross products for orientation testing, and point-in-triangle tests using barycentric coordinates. The trajectory module (`trajectory.py`) defines trajectories as JAX pytrees (enabling automatic differentiation and JIT compilation), storing time-parameterized position and velocity data with linear interpolation for querying arbitrary time-points, path length computation via cumulative Euclidean distances, partial trajectory extraction with boundary interpolation, and trajectory concatenation with automatic time shifting. The controller module (`controller.py`) establishes the agent control architecture through an abstract base class (`AgentController`) and three concrete implementations: `SimpleGoalController` (direct goal-seeking with configurable max speed), `WaypointFollower` (trajectory tracking via time-based interpolation), and `ManualController` (external input interface for interactive control). A comprehensive test suite (`tests/`

`test_shared/`) validates all shared components with unit tests for collision detection primitives, geometric calculations, trajectory operations, and controller behaviors.

The simulation framework (`src/simulation/`) has been partially implemented. Configuration management (`config.py`) defines the complete dataclass hierarchy for all simulation parameters—workspace geometry, agent settings, planner hyperparameters, neural network checkpoints, and termination criteria—though YAML loading and validation functions remain as stubs awaiting implementation. The simulation controller (`controller.py`) provides two orchestration modes: a simplified `run_game_with_controllers()` function that is fully operational and accepts any pair of `AgentController` instances for testing and prototyping, and a comprehensive `run_simulation()` function that is scaffolded with detailed pseudocode and TODO markers for integration of the full adversarial pipeline (observer network loading, Adversarial RRT* planning, particle filter belief tracking, and game-theoretic MPC). The visualization subsystem (`visualization.py`) is complete, offering workspace trajectory plotting with obstacle rendering and goal annotations, belief distribution evolution charts over time, inter-agent distance plots with interception threshold visualization, CSV export functions for trajectories and metrics, belief history serialization, and animated visualizations with synchronized workspace and belief panels using matplotlib’s animation framework. Test scaffolding (`tests/test_simulation/`) includes fixtures for sample configurations and workspaces alongside initial test stubs for controller and visualization components.

Remaining implementation work focuses on the agent-specific planning algorithms and neural network components. For the deceptive agent, this includes the Adversarial RRT* planner integrating path cost and learned deception cost, the RNN-based surrogate observer network (to be trained on synthetic optimal trajectories using Equinox), and the deception cost evaluator with neural network inference. For the interceptor agent, implementation of the maximum entropy inverse reinforcement learning module to recover the deceptive agent’s behavioral model, the particle filter for maintaining and updating belief distributions over candidate goals, and the game-theoretic model predictive control solver for computing interception controls remains outstanding. Final integration of all components into the complete adversarial evaluation pipeline, comprehensive end-to-end testing, and the experimental evaluation campaign will follow completion of these agent-specific modules.

C. Procedure

The experimental evaluation is designed to assess both individual agent performance and the adversarial interaction

between the deceptive agent and the interceptor. Experiments will be conducted in a series of controlled scenarios with increasing complexity, enabling systematic analysis of each algorithm’s strengths and failure modes.

1) *Baseline Experiments:* The first set of experiments establishes baseline performance for both agents in isolation. For the deceptive agent, we will compare trajectories generated with varying deception weights $\alpha \in \{0.0, 0.3, 0.5, 0.7, 1.0\}$ in a simple workspace with a single circular obstacle and three candidate goals. When $\alpha = 1.0$, the planner reduces to standard RRT* and should produce near-optimal paths. When $\alpha = 0.0$, the planner focuses purely on deception, potentially at the cost of path efficiency. These experiments will quantify the trade-off between path length and observer confusion across the deception weight spectrum.

For the interceptor agent, baseline experiments will evaluate goal inference accuracy as a function of observation time. The deceptive agent will execute pre-planned trajectories (generated with $\alpha = 0.3$) toward each candidate goal, and the particle filter’s belief distribution will be recorded at regular intervals. We expect the interceptor’s confidence in the true goal to increase monotonically with observation time, though deceptive trajectories should delay convergence compared to optimal trajectories. These experiments will establish a performance ceiling for the interceptor when operating against known deceptive strategies.

2) *Neural Network Training and Validation:* The RNN observer network will be trained on a dataset of 1000 synthetic trajectories (200 per candidate goal) generated using standard RRT* in a fixed workspace configuration. Training will use cross-entropy loss with the Adam optimizer, and performance will be evaluated on a held-out test set of 200 trajectories. We expect classification accuracy to exceed 80% on optimal trajectories, as these exhibit clear directionality toward their respective goals. Accuracy on deceptive trajectories (generated post-training) will be measured to validate that the deception cost function successfully exploits the observer’s weaknesses.

The IRL module will be trained on expert demonstrations collected from the deceptive agent executing 500 trajectories with varying deception weights and goal configurations. We will validate the learned reward function by comparing predicted trajectories (generated by forward-simulating the learned policy) against ground-truth deceptive trajectories. Agreement between predicted and actual trajectories, measured by average endpoint error and dynamic time warping distance, will indicate successful behavioral model recovery.

3) *Adversarial Scenario Experiments:* The core experiments pit the deceptive agent against the interceptor in full adversarial scenarios. A baseline scenario features a 10×10 workspace with two circular obstacles and three candidate goals positioned at the workspace corners. The deceptive agent starts at position (1, 1) with true goal (9, 9), while the

interceptor starts at (9, 1). The deceptive agent plans its full trajectory offline using Adversarial RRT* with $\alpha = 0.3$, and the interceptor updates its belief and replans every $\Delta t = 0.1$ seconds using the particle filter and MPC controller.

Success criteria are defined as follows. The deceptive agent succeeds if it reaches its goal (within radius 0.5) before being intercepted. The interceptor succeeds if it approaches within the interception threshold (0.5 units) of the deceptive agent before the goal is reached. Experiments will be repeated across 50 randomized initial configurations to obtain statistically meaningful success rates for each agent.

We will conduct parameter sweeps over key variables: deception weight $\alpha \in \{0.0, 0.1, 0.2, \dots, 1.0\}$, number of candidate goals $M \in \{3, 5, 7\}$, workspace complexity (varying number and size of obstacles), and interceptor advantage (velocity ratio $v_I/v_D \in \{1.0, 1.25, 1.5\}$). These experiments will reveal how deception effectiveness scales with problem difficulty and identify parameter regimes where each agent dominates.

4) *Ablation Studies:* Ablation experiments will isolate the contribution of individual components. To assess the value of deception, we will compare the deceptive agent (with trained observer, $\alpha = 0.3$) against a baseline agent using standard RRT* ($\alpha = 1.0$). We expect the deceptive agent to achieve higher success rates against the interceptor, demonstrating that trajectory obfuscation provides a measurable advantage.

To evaluate the interceptor’s IRL-based prediction, we will compare the full interceptor (with learned behavioral model) against a simplified variant that assumes the deceptive agent follows optimal paths. This ablation will quantify the benefit of accounting for deceptive behavior in the interceptor’s motion model. Similarly, we will compare the particle filter approach against a maximum likelihood goal estimator that does not maintain a belief distribution, assessing whether probabilistic inference improves robustness to ambiguous observations.

5) *Performance Metrics:* All experiments will collect the following metrics. For the deceptive agent: observer classification accuracy $P(g^* | \xi_D)$ at trajectory completion, path length ratio $J_{\text{path}}(\xi_D^*)/J_{\text{path}}(\xi_D^{\text{opt}})$ comparing actual path length to the optimal path, and success rate (percentage of trials reaching goal before interception). For the interceptor: goal inference accuracy (whether MAP estimate equals true goal), belief entropy $H(b_t)$ over time, time to confident inference (first time $\max_g b_t(g) > 0.8$), and interception success rate. For the adversarial interaction: minimum distance between agents during execution, time to interception (if successful), and computational performance (planning time, MPC solve time per step).

These metrics will be aggregated across trials and analyzed using statistical hypothesis tests (Mann-Whitney U test for success rates, t-tests for continuous metrics) to determine whether observed differences are statistically significant.

REFERENCES

- [1] H. Nichols, M. Jimenez, Z. Goddard, M. Sparapany, B. Boots, and A. Mazumdar, "Adversarial Sampling-Based Motion Planning," *IEEE Robotics and Automation Letters*, vol. 7, no. 2, pp. 4267–4274, 2022, doi: 10.1109/LRA.2022.3148464.
- [2] K. Xu, Y. Zeng, L. Qin, and Q. Yin, "Single Real Goal, Magnitude-Based Deceptive Path-Planning," *Entropy*, vol. 22, no. 1, p. 88, 2020, doi: 10.3390/e22010088.
- [3] Y. Zeng and K. Xu, "Recognition and interfere deceptive behavior based on inverse reinforcement learning and game theory," *Journal of Systems Engineering and Electronics*, vol. 34, no. 2, pp. 270–288, 2023, doi: 10.23919/JSEE.2023.000012.
- [4] B. Tastan, Y. Chang, and G. Sukthankar, "Learning to intercept opponents in first person shooter games," in *2012 IEEE Conference on Computational Intelligence and Games (CIG)*, 2012, pp. 100–107. doi: 10.1109/CIG.2012.6374144.
- [5] K. Xu and Q. Yin, "Goal Identification Control Using an Information Entropy-Based Goal Uncertainty Metric," *Entropy*, vol. 21, no. 3, p. 299, 2019, doi: 10.3390/e21030299.
- [6] J. L. V. Espinoza, A. Liniger, W. Schwarting, D. Rus, and L. V. Gool, "Deep Interactive Motion Prediction and Planning: Playing Games with Motion Prediction Models," in *Proceedings of The 4th Annual Learning for Dynamics and Control Conference*, R. Firoozi, N. Mehr, E. Yel, R. Antonova, J. Bohg, M. Schwager, and M. Kochenderfer, Eds., in *Proceedings of Machine Learning Research*, vol. 168. PMLR, 2022, pp. 1006–1019. doi: 10.48550/arXiv.2204.02392.
- [7] J. Netter and K. G. Vamvoudakis, "Motion Planning with Adversary Avoidance Using Gaussian Process Classification," in *2024 IEEE 63rd Conference on Decision and Control (CDC)*, 2024, pp. 2299–2304. doi: 10.1109/CDC56724.2024.10886561.
- [8] S. Le Cleac'h, M. Schwager, and Z. Manchester, "LUCIDGames: Online Unscented Inverse Dynamic Games for Adaptive Trajectory Prediction and Planning," *IEEE Robotics and Automation Letters*, vol. 6, no. 3, pp. 5485–5492, 2021, doi: 10.1109/LRA.2021.3074880.